

What changed in the engine since your version

Your version: 80f050e2 (14 Jun). Current: 5ce1a76c (8 Jul). Below is what changed, in plain terms, with the numbers your Go code should reproduce. The file `engine/verify/lens_selfcheck.js` (41 checks) tests every number here — run your port against it.

Tag key: **PORT** = changes the numbers, you must implement it. **UI** = front-end only, skip unless you mirror the display.

Math you must reproduce

1. In-the-money option prices were wrong — now fixed. **PORT**

Below the strike, your version priced options for *less* than they were actually worth, and it put the "exercise now" line in the wrong place. Both fixed.

Reproduce (put, strike \$100, steepness $y=2$): the exercise line sits at **\$66.67**, value there = **1/3**; value at-the-money = **0.148**. With the steepness knob $m=3$: exercise line **\$85.71**, value **1/7**, at-the-money **0.057**.

Hard rule (must always hold): an option is never priced below what you'd get by exercising it.

2. A trade now bends the curve at the trade's own spot, not at the pool's middle. **PORT**

When a trade happens away from the middle of the pool, apply the pool formula *at that spot*. Your version applied it at the middle — that was a bug we agreed to fix.

Reproduce: pool = (10 units, 10 cash, weight 0.5), trade at ray 4, cash in = 1 → new weight = **11/21** (your old code would give 22/43 — wrong). Ordinary spot trades are unchanged.

3. One steepness knob m . **PORT**

A single number sets how steep the option curve is: $m=1$ = normal, bigger m = steeper everywhere (and trades land further out). Set once from the asset's volatility. (Skip if your line already has this.)

The new close — the big change

4. Closing a position is now one simple rule: sell every leg back to the pool at today's price. **PORT**

Before, closing had **two cases**: normal (out-of-the-money) legs traded back through the pool, but in-the-money legs were cashed out separately — and the two didn't line up, so a trader's payout could **jump by roughly half** right at the strike. Now **every** leg, in or out of the money, just sells back through the pool the same way. The payout moves **smoothly** across the strike — no jump.

Key point for your accounting: what the trader is **paid** = the option's value, calculated *before* the pool trade. The pool trade itself is just bookkeeping — it never pays the trader anything extra. (This is what makes it impossible to game — see the warning below.)

5. Funding is being reworked — this version ships the correct DEVIATION (the input), not the final funding number yet. **PORT**

Your version's funding was wrong: it charged funding even on a perfectly **balanced** pool. Fixed to the correct measure — **the ray deviation between the pool curve and a balanced (anchor) curve, at your strike**.

Concretely: take the slope of the live pool curve at the strike, find where the balanced (symmetric) curve has that **same slope**, and compare the two ray angles — that is the **curve skew / ray deviation**. It is **zero when**

the pool is balanced, zero at-the-money, zero in-the-money, and grows the more the pool curve is skewed — a smooth out-of-the-money lobe.

Intended purpose — the same as a perp's funding (a tether / imbalance signal, not a volatility premium). In the planned formula the **pool-curve ray angle is the "mark" and the anchor-curve ray angle is the "oracle"**, so this deviation is the mark-vs-oracle gap that the funding rate will be computed from. **This deviation is the INPUT only — not the funding rate, and not the cash transfer**; both the rate formula (Hyperliquid-style, with a cap) and the actual funding transfer are the **NEXT update**. So this version shows the deviation as a placeholder (the UI column is labeled Funding (ray dev; TBD)).

Port now: the deviation $dev = |c \cdot \ln(strike/mode)|$, $c = (g_anchor - g)/(g_anchor + 1)$, $g = m \cdot \gamma$ (pool), $g_anchor = m$ (balanced), gated to zero in-the-money. **Port next update:** the deviation $\rightarrow$ rate formula + cap. Sign (who pays) is the wing sign, unchanged. **Test:** on a balanced pool this must read zero at every strike — that's the check that catches the old bug.

⚠ Warning — read before you run this on a real shared pool.

The new close does **not** perfectly restore the pool. After a position opens and closes, the pool ends up with a **tiny bit less** than it started.

- **Nobody can steal it.** The trader is only ever paid the option's value — never the pool's leftover. We verified this in the code (there is no path that pays the trader the pool's reserves).
- **It's tiny and it recovers** — like normal impermanent loss. If the price leaves and comes back, it recovers. Size is roughly proportional to trade-size² (a few dollars on a normal trade).
- **In the single-user demo it's harmless** (you'd only shortchange your own pool). **In a real multi-party pool it's a small drift that the liquidity providers bear.**

Before you ship the shared/multi-party version, add "update-2" — a small charge that cancels this drift and restores the exact guarantee. It's already designed (`specs/FIX_close_b_receipt_charge_PARKED_2026-07-03.md`); we just haven't built it into this version yet. Don't run the shared pool without it.

Display only — skip unless you mirror the UI

UI Chart 2: true option-value wings both sides, crossing at the money; fraction/dollar toggle. **UI Vol caption:** fixed — MORE volatile = LOWER m. **UI m input** clamped to 1–6. **UI Funding column** in the portfolio (signed: negative = the line paid; shown P/L includes funding, cash at close still settles without it for now). **UI caption** fixes.

How to check your port

- Run `engine/verify/lens_selfcheck.js` — 41 automated checks; every number above is asserted, and each check is built to fail if you get it wrong. This is your acceptance test.
- The old "the pool always returns to exactly where it started" check was removed (the new close doesn't do that — see the warning); it comes back with update-2.
- The core results are also proved in Lean (by Harmonic's Aristotle prover) — proofs available on request.

Not in this version yet (so nothing surprises you)

- **Update-2:** the safety charge that cancels the pool drift (needed before multi-party).
- Funding cash actually moving between the two sides (right now it's only calculated/shown, not transferred).

- Smoothing of the funding/price reads over time (a later tuning pass).

Latest engine file: `engine/builds/HEAD_temporal_mvp_v28_lens.html` (md5 `5ce1a76c` ; this dir: `temporal_mvp_v28_lens_5ce1a76c.html`). The previous close is kept as a backup: `temporal_mvp_v28_lens_twocaseclose.html` . Generated 8 Jul 2026.